

Atividades

```
import unittest
from unittest.mock import patch, MagicMock
```

1. Função is_par

```
def is_par(n: int) -> bool:
    return n % 2 == 0
```

2. Função fatorial

```
def fatorial(n: int) -> int:
    if n < 0:
        raise ValueError("Número negativo não permitido")
    if n == 0:
        return 1
    resultado = 1
    for i in range(1, n + 1):
        resultado *= i
    return resultado
```

3. Classe Conta

```
class InsufficientFunds(Exception):
    pass
```

```
class Conta:
    def __init__(self, saldo=0):
        self.saldo = saldo

    def depositar(self, amount: float):
        if amount < 0:
            raise ValueError("Depósito não pode ser negativo")
        self.saldo += amount

    def sacar(self, amount: float):
        if amount < 0:
            raise ValueError("Saque não pode ser negativo")
        if amount > self.saldo:
```

```
        raise InsufficientFunds("Saldo insuficiente")
    self.saldo -= amount
```

4. Função buscar_clima

```
import requests
```

```
def buscar_clima(cidade: str) -> float:
    url = f"https://api.exemplo/clima?cidade={cidade}"
    response = requests.get(url)
    data = response.json()
    if "temperatura" not in data:
        raise ValueError("Resposta inválida da API")
    return data["temperatura"]
```

TESTES UNITÁRIOS

```
class TestFuncoes(unittest.TestCase):
```

```
    # ----- Testes is_par -----
```

```
    def test_is_par_true(self):
        self.assertTrue(is_par(4))
```

```
    def test_is_par_false(self):
        self.assertFalse(is_par(5))
```

```
    def test_is_par_zero(self):
        self.assertTrue(is_par(0))
```

```
    def test_is_par_negativo(self):
        self.assertTrue(is_par(-2))
        self.assertFalse(is_par(-3))
```

```
    def test_fatorial_zero(self):
        self.assertEqual(fatorial(0), 1)
```

```
    def test_fatorial_positivo(self):
        self.assertEqual(fatorial(5), 120)
```

```
    def test_fatorial_negativo(self):
```

```

        with self.assertRaises(ValueError):
            fatorial(-1)

def test_deposito_valido(self):
    conta = Conta()
    conta.depositar(100)
    self.assertEqual(conta.saldo, 100)

def test_deposito_invalido(self):
    conta = Conta()
    with self.assertRaises(ValueError):
        conta.depositar(-50)

def test_saque_valido(self):
    conta = Conta(200)
    conta.sacar(150)
    self.assertEqual(conta.saldo, 50)

def test_saque_insuficiente(self):
    conta = Conta(100)
    with self.assertRaises(InsufficientFunds):
        conta.sacar(200)

def test_saque_invalido(self):
    conta = Conta(100)
    with self.assertRaises(ValueError):
        conta.sacar(-10)

@patch("requests.get")
def test_buscar_clima_valido(self, mock_get):
    mock_response = MagicMock()
    mock_response.json.return_value = {"temperatura": 25}
    mock_get.return_value = mock_response

    temp = buscar_clima("Curitiba")
    self.assertEqual(temp, 25)

@patch("requests.get")
def test_buscar_clima_sem_temperatura(self, mock_get):
    mock_response = MagicMock()
    mock_response.json.return_value = {"umidade": 80}
    mock_get.return_value = mock_response

```

```
        with self.assertRaises(ValueError):
            buscar_clima("São Paulo")

if __name__ == "__main__":
    unittest.main(argv=[""], exit=False)
```